Java Streams and Caffeine Dreams

Mikaël Francoeur

Quiz: What does this print?

```
[1, 2, 3, 4, 5].map(i => {
    console.log(i);
    return i;
});
```

It prints `1 2 3 4 5`

Quiz: What does this print? (v2)

```
Stream.of(1, 2, 3, 4, 5)
        .map(i -> {
            IO.println(i);
            return i;
        });
```

- It prints **nothing**!

Quiz: What does this print? (v3)

```
Stream.of(1, 2, 3, 4, 5)
        .peek(System.out::println)
        .count();
```

- It prints **nothing**!

Quiz: What does this print? (v4)

```
Stream.of(1, 2, 3, 4, 5)
        .filter(i -> true)
        .peek(System.out::println)
        .count();
```

- It prints `1 2 3 4 5`

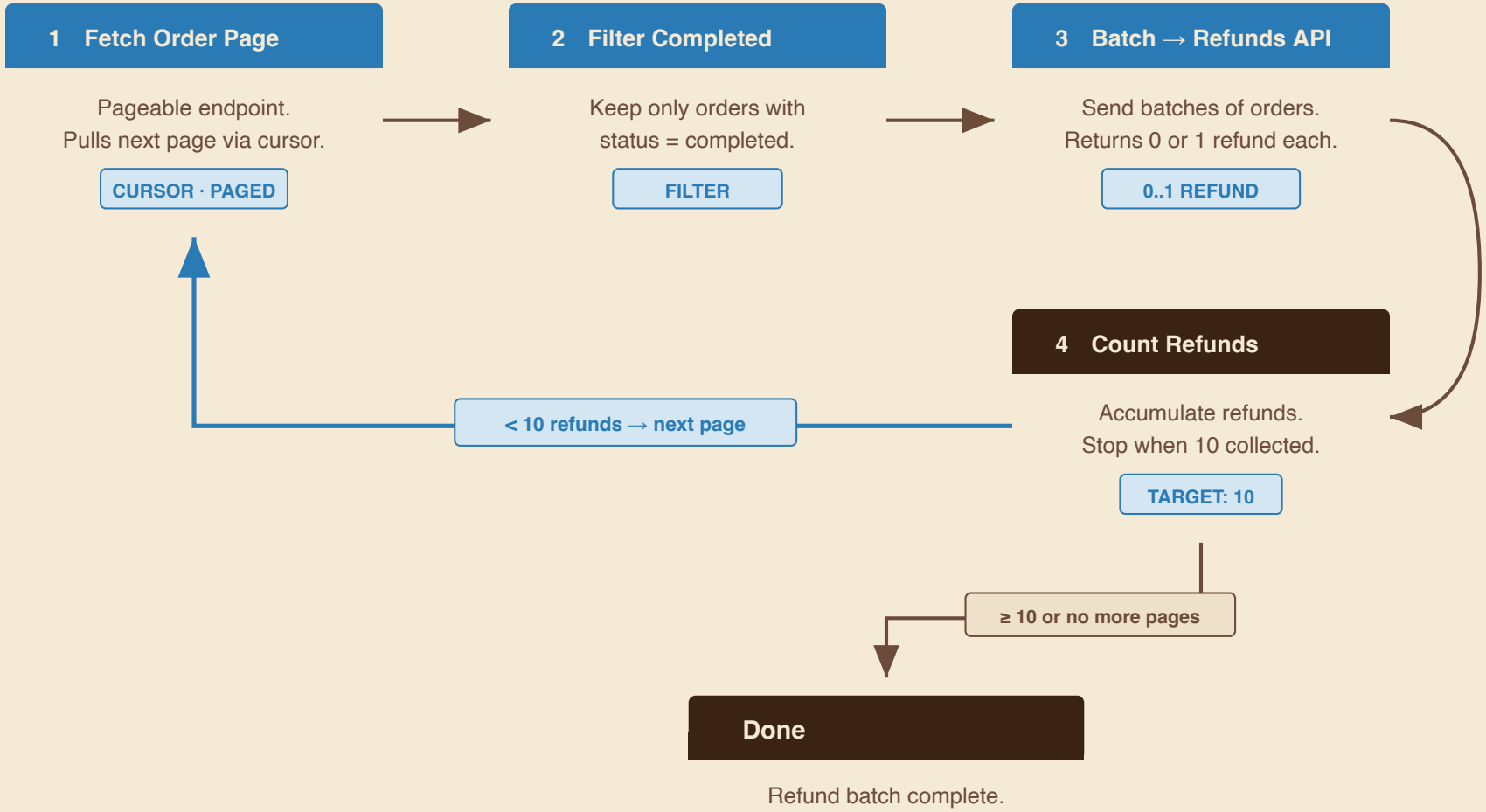
Quiz: what does this print?

```
Stream.of(1, 2, 3, 4, 5)
        .map(i -> CompletableFuture.supplyAsync(() -> {
            sleepOneSecond();
            IO.println(i);
            return i;
        }, Executors.newVirtualThreadPerTaskExecutor()))
        .map(CompletableFuture::join)
        .toList();
```

- It prints `1 2 3 4 5`, one number every second!

Processing Refunds

Processing Refunds



GOAL: Only consume as many pages from the Orders API as needed

Conclusion

- 3 types of operation
- Streams are lazy (1->map->print, 2->map->print, 3->…)
- Operations communicate metadata downstream and cancellation upstream
- Gatherers for concurrency (blocking) and batching
- Real-world application

Thank you!

Bonus Slides

Spliterator interface

```
public interface Spliterator<T> {

    default void forEachRemaining(Consumer<? super T> action);
    boolean tryAdvance(Consumer<? super T> action);

    Spliterator<T> trySplit();

    long estimateSize();
    default long getExactSizeIfKnown() {...}

    int characteristics();
    default boolean hasCharacteristics(int characteristics) {...}

    default Comparator<? super T> getComparator() {...}

}
```

Quiz: how long does this take?

```
IntStream.range(0, 100)
        .boxed()
        .parallel()
        .peek(i -> {
            sleepOneSecond();
            IO.println(i);
        })
        .toList();
```

- It **depends**!

Gatherers

- Java 24
- intermediate operations
- stateless or stateful
- can short-circuit
- builtin: `mapConcurrent`, `windowFixed`, `windowSliding`, `scan`, `fold`

Link to the Orders and Refunds demo

https://github.com/LeMikaelF/confoo-2026-streams